AMBA APB (ADVANCED PERIPHERAL BUS) DESIGN AND VERIFICATION

Verilog Based APB Design

By

Manmohan verma

Contents

1. Introduction 2

1.1 AMBA APB Overview 2

2. APB Architecture 3

2.1 APB Block Diagram 3

2.2 APB Signals 4

2.2.1 Clock and Reset 4

2.2.2 Control Signals 4

2.2.3. Address and Data Signals 5

2.2.4 Status Signals 5

3. Design Implementation 5

3.1 APB Master 5

3.2 APB Slave 6

3.3 Top Module 6

4. Verilog Code 7

4.1 APB Master Code 7

4.2 APB Slave Code 10

4.3 Top Module Code: 13

4.4 Testbench Code 15

5. Simulation Results 17

5.1 Write Operation Waveform 17

5.2 Read Operation Waveform 18

6. RTL Schematic 19

7. Tools Used 20

8. Conclusion 20

1. Introduction

1.1 AMBA APB Overview

AMBA (Advanced Microcontroller Bus Architecture) is a standard bus protocol developed by ARM for communication between different modules inside a System-on-Chip (SoC).

In any digital system, multiple components such as processor, memory, and peripherals need to communicate with each other. AMBA provides a structured way to perform this communication.

APB (Advanced Peripheral Bus) is one of the simplest buses in the AMBA family. It is specifically designed for communication with low-speed peripherals such as UART, Timer, GPIO, and SPI.

Unlike high-performance buses like AHB or AXI, APB does not support:

- Pipelining
- Burst transfers

Instead, it performs simple one-by-one data transfers, which makes it:

- Easy to design
- Power-efficient
- Suitable for simple devices

Why APB is used:

- Low complexity reduces hardware cost
- Less switching activity → low power consumption
- Ideal for control-based peripherals (not data-heavy tasks)

1.2 Project Objective

The main objective of this project is to design and verify the APB protocol using Verilog HDL.

This project focuses on understanding how communication happens between a Master and a Slave using APB protocol.

In this project, we perform:

- Design of APB Master (controls the communication)

- Design of APB Slave (responds to requests)
- Integration using Top Module
- Verification using Testbench

Learning Outcomes:

- Understanding of bus protocols
- FSM-based design implementation
- Timing and control signal handling
- Verification using simulation

2. APB Architecture

2.1 APB Block Diagram

The APB system mainly consists of two components:

i. Master

ii. Slave

The Master initiates all operations, while the Slave responds accordingly.

Step-by-step working:

i. The Master places the address on the address bus (PADDR)

ii. It selects the slave using PSEL

iii. It decides operation using PWRITE (read/write)

iv. In the next clock cycle, PENABLE is asserted

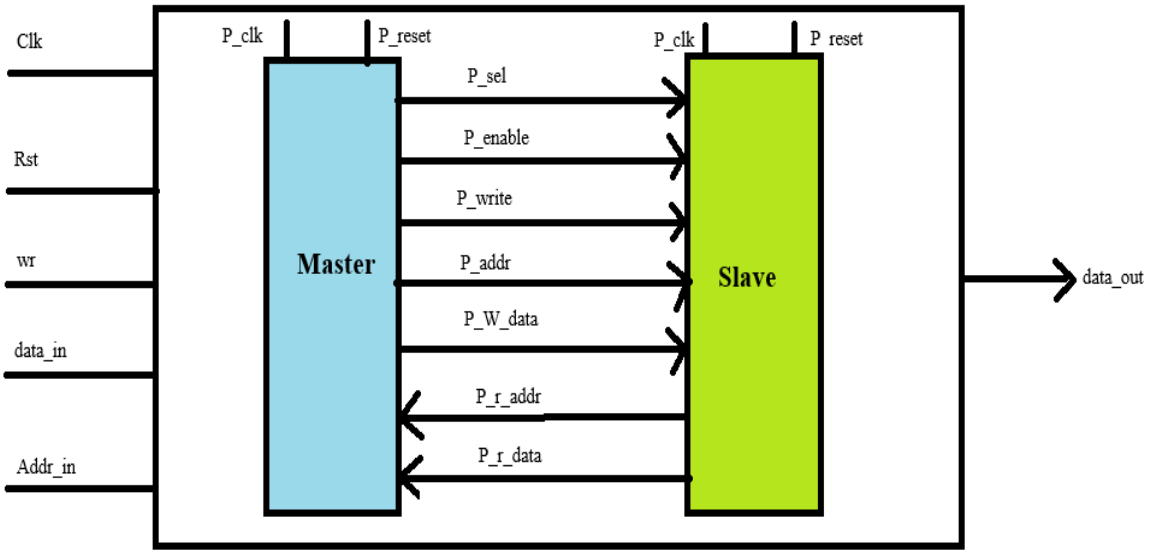
v. Data transfer takes place

vi. Slave signals completion using PREADY

Important concept: Two-phase transfer

- Setup Phase
- Enable Phase

This makes APB timing very simple and predictable.



2.2 APB Signals

2.2.1 Clock and Reset

PCLK (Clock): All operations are synchronized with this clock. Every signal change happens on clock edges

PRESETn (Reset): Active-low reset signal. When this is 0, all registers are reset.

2.2.2 Control Signals

PSEL (Select):

- Used to select a particular slave.
- If PSEL = 0 → no operation
- If PSEL = 1 → transfer starts

PENABLE (Enable):

- Indicates second phase of transfer.
- Works only after PSEL is high.

PWRITE:

- Defines type of operation:
- 1 → Write operation
- 0 → Read operation

2.2.3. Address and Data Signals

PADDR: Carries the address where data will be read or written.

PWDATA: Carries data from Master to Slave (write operation).

PRDATA: Carries data from Slave to Master (read operation).

2.2.4 Status Signals

PREADY: Indicates that the Slave has completed the operation.

If 0 → wait

If 1 → transfer complete

PSLVERR: Indicates error during transfer (optional signal).

3. Design Implementation

3.1 APB Master

The APB Master is responsible for controlling the entire communication process.

It generates:

- Address (PADDR)
- Control signals (PSEL, PENABLE, PWRITE)
- Write data (PWDATA)

FSM (Finite State Machine) is used:

States:

i. IDLE

No operation

Waiting for request

ii. SETUP

PSEL = 1

Address and control signals are set

iii. ENABLE

PENABLE = 1

Data transfer happens

Flow: IDLE → SETUP → ENABLE → IDLE

This sequence repeats for every transfer.

3.2 APB Slave

The APB Slave receives commands from the Master and performs operations.

Key responsibilities:

- Check if it is selected (PSEL)
- Decode address (PADDR)
- Perform read/write operation
- Send response using PREADY

Write Operation:

- Master sends address + data
- Slave stores data in register/memory

Read Operation:

- Master sends address
- Slave reads data and sends via PRDATA

3.3 Top Module

- The Top Module connects Master and Slave.
- It acts as a bridge and ensures all signals are properly connected.

Responsibilities:

- Signal routing
- System integration
- Complete APB communication system

4. Verilog Code

4.1 APB Master Code

The Master code includes:

- FSM implementation
- Control signal generation
- Address and data handling

Important logic:

- PSEL active in setup phase
- PENABLE active in enable phase
- Signals controlled based on FSM

```
module apb_m(
        input pclk,
        input presetn,
        input [3:0] addrin,
        input [7:0] datain,
        input wr,
        input newd,
        input [7:0] prdata,
        input pready,
        output reg psel,
        output reg penable,
        output reg slverr,
        output reg [3:0] paddr,
        output reg [7:0] pwdata,
        output reg pwrite,
        output [7:0] dataout  //for read operation as combi
        );
         localparam [1:0] idle = 0, setup = 1, enable =2;
        reg [1:0] state, nstate;

        //reset decoder
```

```
 always@(posedge pclk, negedge presetn)
    begin
       if (presetn ==  1'b0)
           state <= idle;
       else
           state <= nstate;
    end

//state decoder
always@(*)
 begin
   case(state)
    idle:
      begin
      if (newd == 1'b0)
         nstate = idle;
      else
         nstate = setup;
      end
    setup:
      begin
       nstate = enable;
      end
    enable:
      begin
       if(newd == 1'b1 )
          begin
            if(pready == 1'b1)
               nstate = setup;
            else
               nstate = enable;
          end
      else
          begin
```

```
                nstate = idle;
              end
            end
          default : nstate = idle;
        endcase
      end

//address decoding
always@(posedge pclk, negedge presetn)
    begin
        if(presetn == 1'b0)
            begin
              psel <= 1'b0;
            end
         else if (nstate == idle)
            begin
              psel <= 1'b0;
            end
         else if (nstate == enable || nstate == setup)
            begin
             psel <= 1'b1;
            end
         else
            begin
             psel <= 1'b0;
            end
    end
 // output logic
 always@(posedge pclk, negedge presetn)
     begin
        if(presetn == 1'b0)
          begin
          penable <= 1'b0;
          paddr   <= 4'h0;
```

```
      pwdata  <= 8'h00;
      pwrite  <= 1'b0;
      end
    else if (nstate == idle)
      begin
      penable <= 1'b0;
      paddr   <= 4'h0;
      pwdata  <= 8'h00;
      pwrite  <= 1'b0;
      end
    else if (nstate == setup)
      begin
        penable <= 1'b0;
        paddr   <= addrin;
        pwrite  <= wr;
         if(wr == 1'b1)
             begin
             pwdata <= datain;
             end
         end
    else if (nstate == enable)
        begin
           penable <= 1'b1;
        end
  end
 assign dataout = (psel == 1'b1 &&  penable == 1'b1 && wr == 1'b0) ? prdata : 8'h00;
endmodule
```

4.2 APB Slave Code

- The Slave code includes:
- Register/memory implementation
- Address decoding
- Read/write logic

Example:

If write → store data

If read → send stored data

Code:

```
`timescale 1ns/1ps
module apb_s(
   input  pclk,
   input  presetn,
   input [3:0] paddr,
   input psel,
   input penable,
   input [7:0] pwdata,
   input pwrite,
   output reg [7:0] prdata,
   output reg pready
);
  localparam [1:0] idle = 0, write = 1, read = 2;
  reg [7:0] mem[0:15];
  reg [1:0] state, nstate;


  //////reset decoder
  always@(posedge pclk, negedge presetn)
    begin
      if(presetn == 1'b0)
          state <= idle;
      else
```

```
        state <= nstate;
    end
//////state decoder
  always@(*)
    begin
    case(state)
     idle:
     begin
       prdata = 8'h00;
       pready = 1'b0;
           if(psel == 1'b1 && pwrite == 1'b1)
               nstate = write;
           else if (psel == 1'b1 && pwrite == 1'b0)
               nstate = read;
           else
               nstate = idle;
     end
     write:
     begin
       if(psel == 1'b1 && penable == 1'b1)
           begin
               pready = 1'b1;
               mem[paddr]  = pwdata;
               nstate      = idle;
           end
       else
```

```
            begin
            nstate      = idle;
            end
     end
    read:
     begin
        if(psel == 1'b1 && penable == 1'b1 )
            begin
                pready = 1'b1;
                prdata = mem[paddr];
                nstate      = idle;
               end
        else
            begin
            nstate      = idle;
            end
     end
    default : nstate = idle;
    endcase
    end
endmodule
```

4.3 Top Module Code:

```
 `timescale 1ns / 1ps
module top_m_s(
input clk,rstn,wr,newd,
input [3:0] ain,
```

```
input [7:0] din,
output [7:0] dout
    );
wire psel, penable, pready, pwrite;
wire [7:0] prdata, pwdata;
wire [3:0] paddr;
apb_m m1 (
        .pclk(clk),
        .presetn(rstn),
        .addrin(ain),
        .datain(din),
        .wr(wr),
        .newd(newd),
        .prdata(prdata),
        .pready(pready),
        .psel(psel),
        .penable(penable),
        .paddr(paddr),
        .pwdata(pwdata),
        .pwrite(pwrite),
        .dataout(dout)
    );
    apb_s s1 (
        .pclk(clk),
        .presetn(rstn),
        .paddr(paddr),
```

```
        .psel(psel),
        .penable(penable),
        .pwdata(pwdata),
        .pwrite(pwrite),
        .prdata(prdata),
        .pready(pready)
    );
Endmodule
```

4.4 Testbench Code

Testbench is used to verify functionality.

It performs:

- Clock generation
- Reset generation
- Applying test cases

Test cases include:

- Write data to slave
- Read data from slave
- Verify correctness

Code:

```
        `timescale 1ns / 1ps
module tb_top;
    // Define testbench ports
    reg clk = 0;
    reg rstn;
    reg wr;
    reg newd;
```

```
    reg [3:0] ain;
    reg [7:0] din;
    wire [7:0] dout;
always #10 clk = ~clk;
    // Instantiate the top module
    top_m_s dut (
        .clk(clk),
        .rstn(rstn),
        .wr(wr),
        .newd(newd),
        .ain(ain),
        .din(din),
        .dout(dout)
    );
    initial
    begin
    rstn = 0;
    repeat(5) @(posedge clk);
    rstn = 1;
    newd = 1;
    for(int i = 0; i < 10; i++)
    begin
    ain = $urandom;
    din = $urandom;
    wr = 1;
    @(posedge dut.m1.pready);
```

```
    end
    for(int i = 0; i < 10; i++)
    begin
    ain = $urandom;
    din = 0;
    wr = 0;
    @(posedge dut.m1.pready);
    end
    end
    initial begin
    #900;
    $stop;
    end
endmodule
```

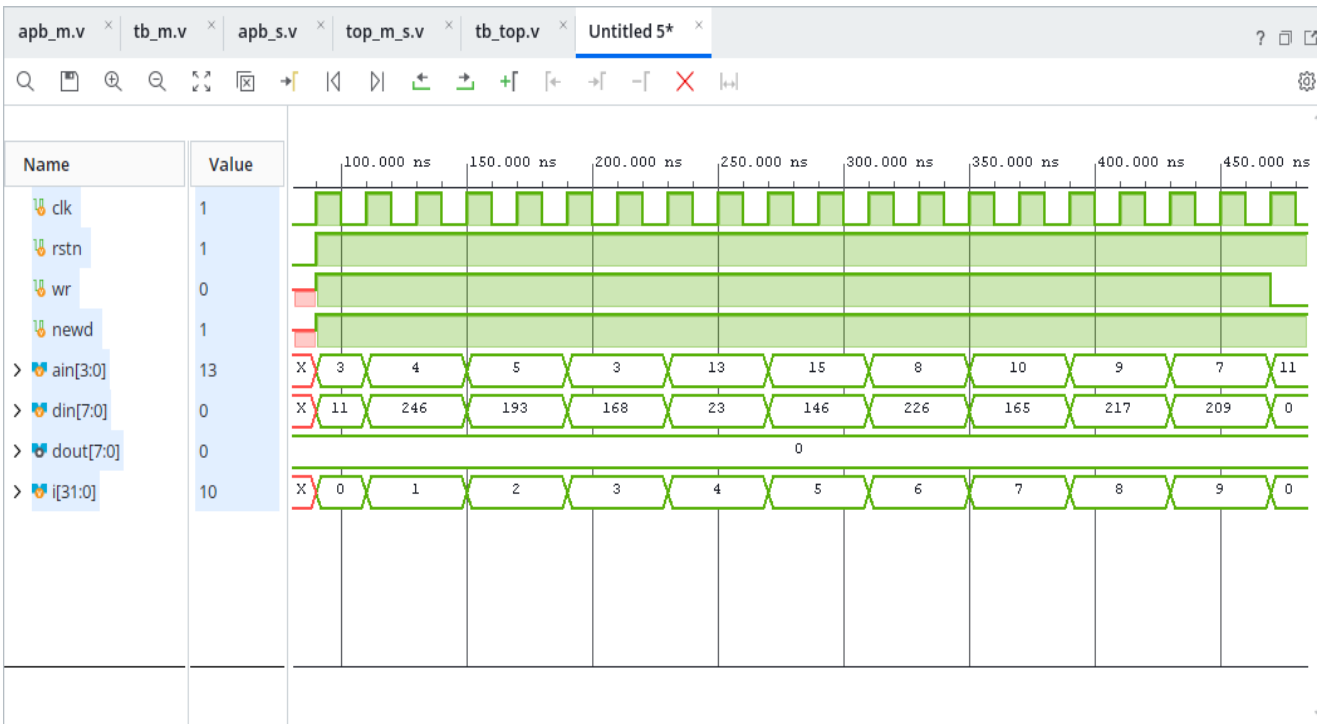
5. Simulation Results

5.1 Write Operation Waveform

In write operation:

- PWRITE = 1
- Address and data are valid in setup phase
- Data is written in enable phase
- Important observation: Data transfer happens only when:

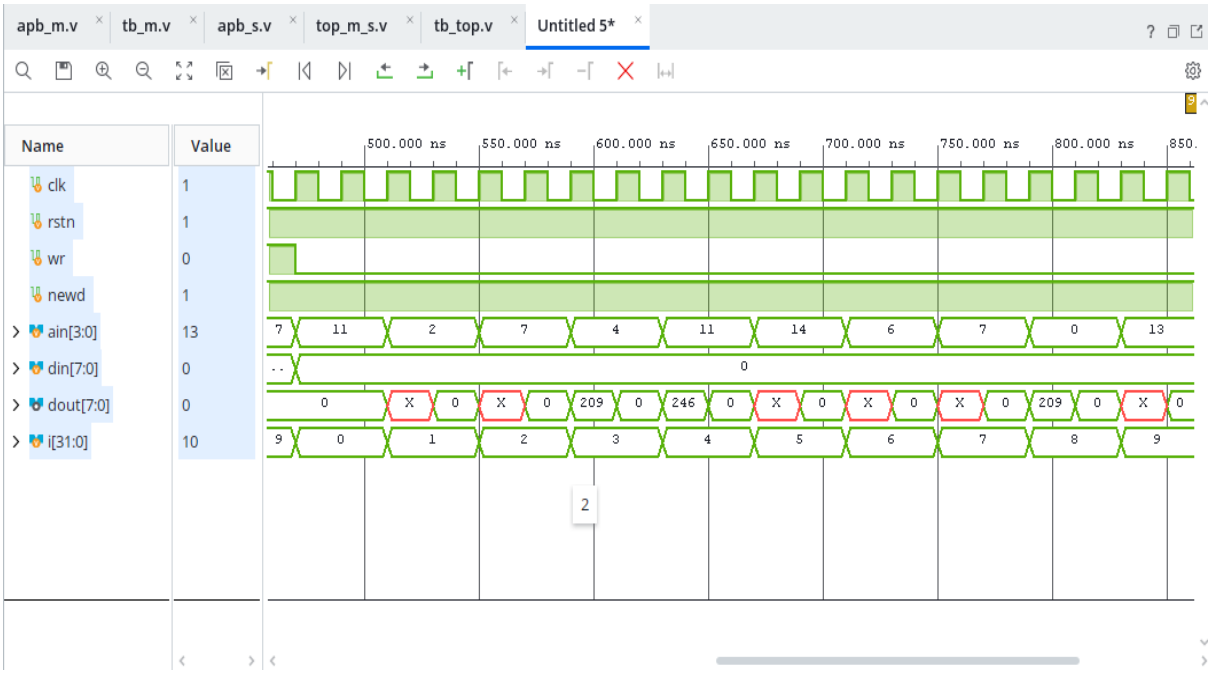
PSEL = 1, PENABLE = 1



5.2 Read Operation Waveform

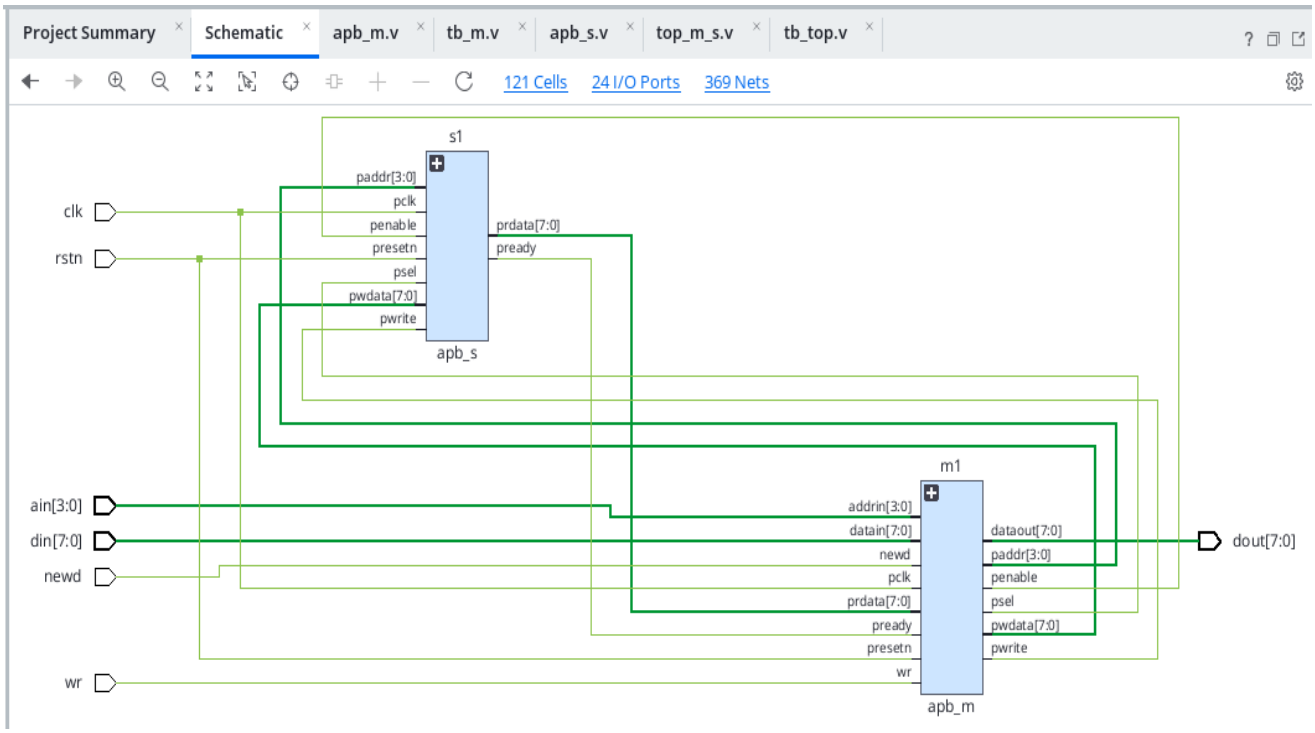
In read operation:

- PWRITE = 0
- Slave sends data using PRDATA
- Important observation:
- Master captures data when PREADY = 1

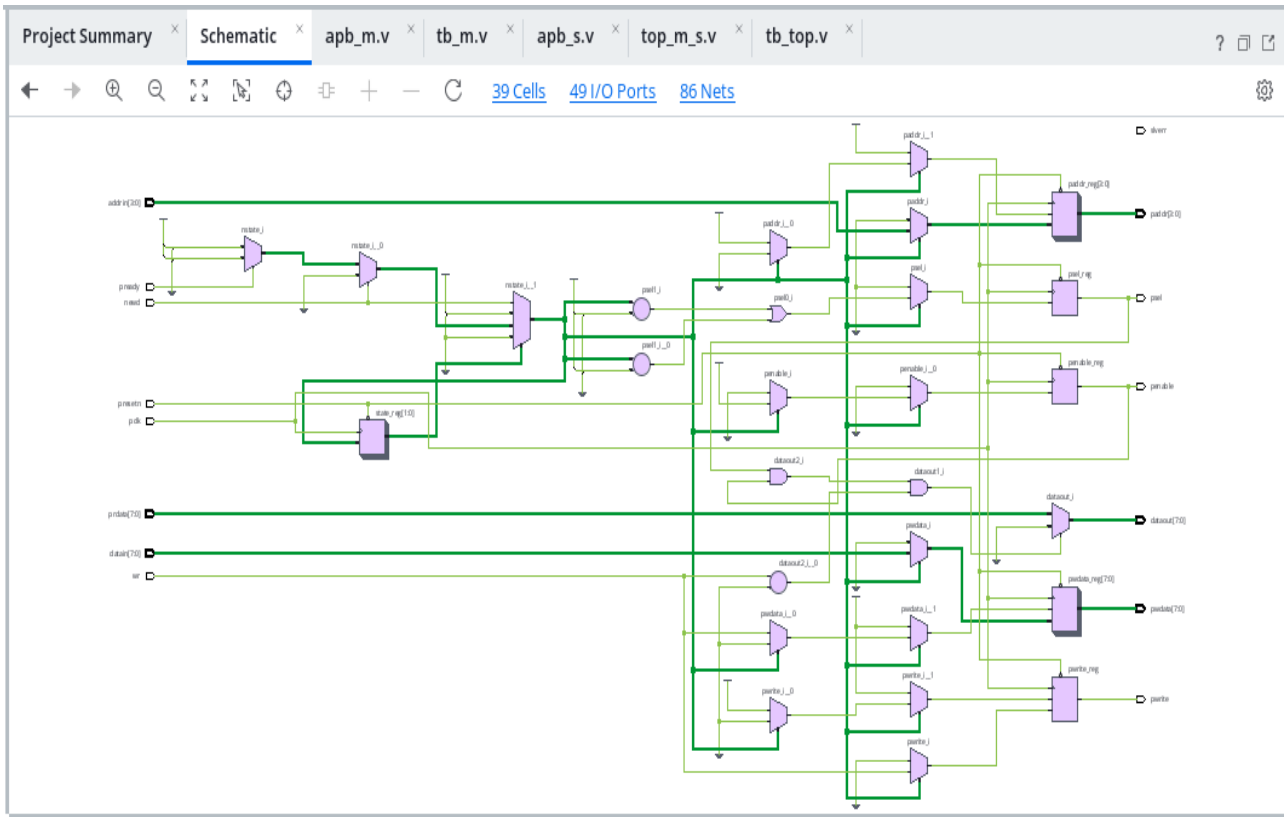


6. RTL Schematic

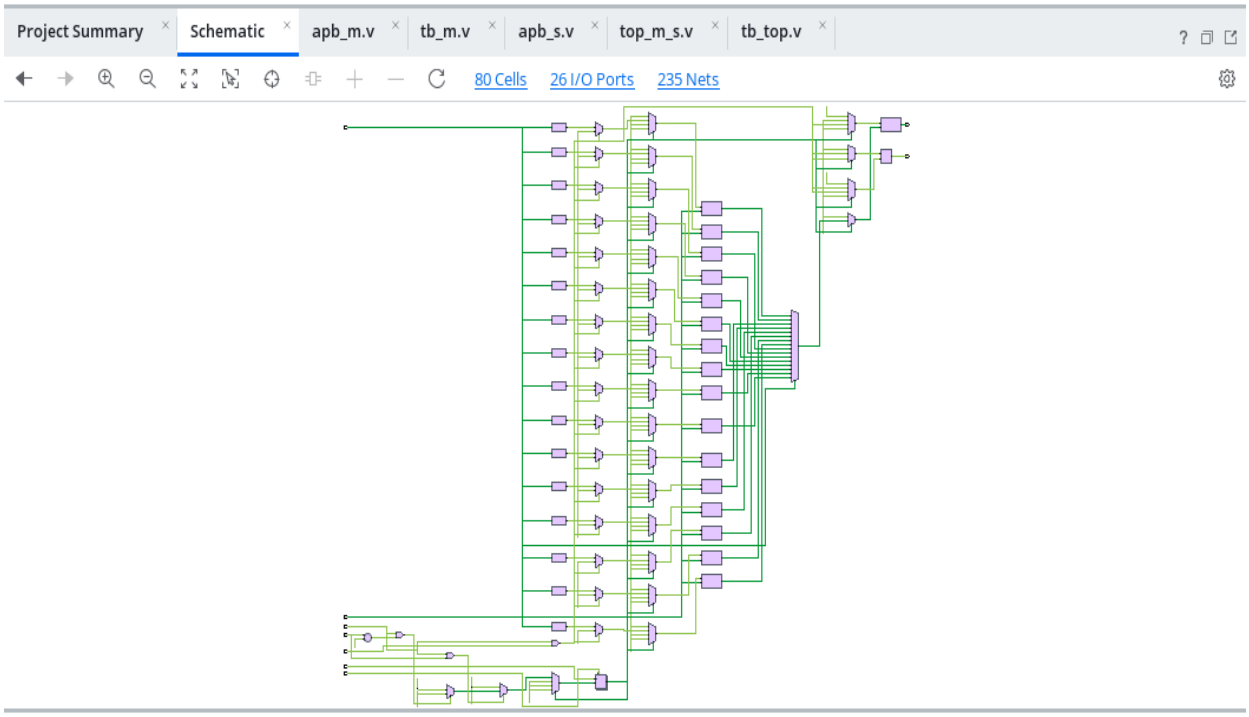
Top module:



Master:



Slave:



7. Tools Used

- Xilinx Vivado → Design and simulation
- Verilog HDL → Coding language

8. Conclusion

In this project, we successfully implemented the APB protocol.

We learned:

- How Master and Slave communicate
- How FSM controls data transfer
- How to verify design using testbench

Final result:

- Correct read/write operations achieved
- Protocol timing verified